

tensors_languages

pyttb

```
#reticulate::py_install(c("numpy", "pyttb"))
```

```
from __future__ import annotations
import sys
import numpy as np
import pyttb as ttb
```

Dense Tensors

Tensor Types

pyttb has five tensor types: dense, sparse, tucker, kruskal, and sum of structured tensors. Dense tensors are analogous to my tensor class, as they are a class that builds on Python's array.

```
M = np.ones((2, 4, 3)) # A 2x4x3 array.
X = ttb.tensor(M) # Convert to a tensor object
X
```

```
tensor of shape (2, 4, 3) with order F
data[:, :, 0] =
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
data[:, :, 1] =
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

```
data[:, :, 2] =
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

Reshape

Similar to `aperm`, they have a function called `reshape`.

```
X = X.reshape((4, 2, 3))
X
```

```
tensor of shape (4, 2, 3) with order F
data[:, :, 0] =
[[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]
data[:, :, 1] =
[[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]
data[:, :, 2] =
[[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]
```

`permute` specifically reorders the modes.

```
X = ttb.tensor(np.arange(1, 25), shape=(2, 3, 4))
print(f"X is a {X}")

X.permute(np.array((2, 1, 0))) # Reverse the modes.
```

```
X is a tensor of shape (2, 3, 4) with order F
data[:, :, 0] =
[[1 3 5]
 [2 4 6]]
data[:, :, 1] =
```

```

[[ 7  9 11]
 [ 8 10 12]]
data[:, :, 2] =
[[13 15 17]
 [14 16 18]]
data[:, :, 3] =
[[19 21 23]
 [20 22 24]]

tensor of shape (4, 3, 2) with order F
data[:, :, 0] =
[[ 1  3  5]
 [ 7  9 11]
 [13 15 17]
 [19 21 23]]
data[:, :, 1] =
[[ 2  4  6]
 [ 8 10 12]
 [14 16 18]
 [20 22 24]]

```

Attributes

You can obtain the original data using `.data`

```
X.data
```

```

array([[[ 1,  7, 13, 19],
        [ 3,  9, 15, 21],
        [ 5, 11, 17, 23]],

       [[ 2,  8, 14, 20],
        [ 4, 10, 16, 22],
        [ 6, 12, 18, 24]]])

```

You can obtain the number of dims with `ndims` and the number of modes using `shape`.

```
X.ndims
```

```
X.shape
```

```
(2, 3, 4)
```

It is possible to create an empty tensor.

```
X = ttb.tensor()  
X
```

```
empty tensor of shape ()  
data = []
```

Creating Tensors

tenones produces a tensor containing only 1.

```
X = ttb.tenones((2, 3, 4))  
X
```

```
tensor of shape (2, 3, 4) with order F  
data[:, :, 0] =  
[[1. 1. 1.]  
 [1. 1. 1.]  
data[:, :, 1] =  
[[1. 1. 1.]  
 [1. 1. 1.]  
data[:, :, 2] =  
[[1. 1. 1.]  
 [1. 1. 1.]  
data[:, :, 3] =  
[[1. 1. 1.]  
 [1. 1. 1.]
```

tenzeroes produces a tensor containing only 0.

```
X = ttb.tenzeros((2, 3, 4))  
X
```

```
tensor of shape (2, 3, 4) with order F  
data[:, :, 0] =  
[[0. 0. 0.]  
 [0. 0. 0.]  
data[:, :, 1] =  
[[0. 0. 0.]
```

```

[0. 0. 0.]
data[:, :, 2] =
[0. 0. 0.]
[0. 0. 0.]
data[:, :, 3] =
[0. 0. 0.]
[0. 0. 0.]

```

tenrand produces a tensor with random elements.

```

np.random.seed(0)
X = ttb.tenrand((2, 3, 4))
X

```

```

tensor of shape (2, 3, 4) with order F
data[:, :, 0] =
[[0.5488135  0.60276338 0.4236548 ]
 [0.71518937 0.54488318 0.64589411]]
data[:, :, 1] =
[[0.43758721 0.96366276 0.79172504]
 [0.891773   0.38344152 0.52889492]]
data[:, :, 2] =
[[0.56804456 0.07103606 0.0202184 ]
 [0.92559664 0.0871293  0.83261985]]
data[:, :, 3] =
[[0.77815675 0.97861834 0.46147936]
 [0.87001215 0.79915856 0.78052918]]

```

squeeze removes modes of size 1

```

X = ttb.tenrand((2, 5, 4))
Y = X.copy()

# reshapes from 2 x 5 x 4 to 2 x 5 x 4 x 1
Y[0, 0, 0, 0] = Y[0, 0, 0]

# removes all modes of size 1
Y.squeeze()

```

```

tensor of shape (2, 5, 4) with order F
data[:, :, 0] =

```

```

[[0.11827443 0.14335329 0.52184832 0.26455561 0.45615033]
 [0.63992102 0.94466892 0.41466194 0.77423369 0.56843395]]
data[:, :, 1] =
[[0.0187898 0.61209572 0.94374808 0.3595079 0.6976312 ]
 [0.6176355 0.616934 0.6818203 0.43703195 0.06022547]]
data[:, :, 2] =
[[0.66676672 0.21038256 0.31542835 0.57019677 0.98837384]
 [0.67063787 0.1289263 0.36371077 0.43860151 0.10204481]]
data[:, :, 3] =
[[0.20887676 0.65310833 0.46631077 0.15896958 0.65632959]
 [0.16130952 0.2532916 0.24442559 0.11037514 0.13818295]]

```

You can convert from tensor to array using double.

```

X = ttb.tenrand((2, 5, 4)) # tensor
X.double() # array

```

```

array([[[[0.19658236, 0.73926358, 0.56660145, 0.28940609],
          [0.82099323, 0.28280696, 0.52324805, 0.58651293],
          [0.83794491, 0.2961402 , 0.5759465 , 0.82894003],
          [0.97645947, 0.31798318, 0.31856895, 0.67781654],
          [0.97676109, 0.0641475 , 0.13179786, 0.73519402]],

        [[0.36872517, 0.03918779, 0.26538949, 0.18319136],
          [0.09710128, 0.12019656, 0.09394051, 0.02010755],
          [0.09609841, 0.11872772, 0.9292962 , 0.00469548],
          [0.4686512 , 0.41426299, 0.66741038, 0.27000797],
          [0.60484552, 0.69247212, 0.7163272 , 0.96218855]]]])

```

Subsetting

Subsetting uses the square brackets similar to R. Using the : keeps all elements from that mode.

```

X = ttb.tenrand((2, 3, 4, 1))
X[0, 0, 0, 0]

X[0, :, 0, :]

```

tensor of shape (3, 1) with order F

```
data[:, :] =
[[0.24875314]
 [0.59204193]
 [0.22308163]]
```

Assignment

For assignment, you can assign a single element, subtensor, or list of values for a tensor.

```
X = ttb.tenrand((2, 3, 4))

X[0, 0, 0] = 0
X[0, 0:2, 0:2] = np.ones((2, 2))

X[(0, 0, 0)], X[(1, 0, 0)] = [5, 7]
```

Assigning elements outside the original range expands the tensor.

```
X.shape
X[2, 1, 1] = 1
X.shape
```

```
(np.int64(3), np.int64(3), np.int64(4))
```

Nonzero elements

```
X = ttb.tensor(3 * np.random.rand(2, 2, 2))
S, V = X.find()
S # nonzero subscripts
V # nonzero values
```

```
array([[0.4874788 ],
       [2.42195688],
       [0.37145995],
       [1.22154989],
       [1.84667869],
       [1.70730222],
       [2.54402469],
       [0.20750099]])
```

You can also supply a condition for elements you are looking for.

```
larger_entries = X >= 2
larger_subs, larger_vals = larger_entries.find()
larger_subs, larger_vals
```

```
(array([[1, 0, 0],
        [0, 1, 1]]),
 array([[ True],
        [ True]]))
```

Norm

The Frobenius norm is computed with `norm`.

```
X.norm()
```

```
4.535845968377549
```

Elementwise Operations

pyttb supports logical operations with `logical_and`, `logical_or`, and `logical_xor`.

The method `tenfun` allows any function not predefined to tensors to work on them element-wise.

```
A = ttb.tensor(np.floor(3 * np.random.rand(2, 2, 3)))
A.tenfun(lambda x: x + 1) # add 1 to every element
```

```
tensor of shape (2, 2, 3) with order F
data[:, :, 0] =
[[3. 3.]
 [1. 1.]]
data[:, :, 1] =
[[2. 3.]
 [2. 2.]]
data[:, :, 2] =
[[3. 3.]
 [3. 1.]]
```

Symmetry

A tensor can be symmetrized with `symmetrize`. This is formed by averaging over all elements that should be equal.

```
np.random.rand(0)
X = ttb.tensor(np.arange(1, 5), (4,)) # Create some data
W = ttb.tensor(np.random.rand(4, 4, 4))
Y = X.symmetrize()

Y.issymmetric()
```

True

Sparse Tensors

The `sptensor` class stores data in coordinate format. It can be created with a list of subscripts and values. The size of the tensor is implicitly the size of the largest subscript in each dimension, or it can be explicitly specified.

```
np.random.seed(0)
subs = np.array([[0, 0, 0], [0, 1, 0], [2, 3, 1]]) # Subscripts of the nonzeros.
vals = np.array([[1], [2], [3]]) # Vals is a column vector; values of the nonzeros.
X = ttb.sptensor.from_aggregator(subs, vals) # Sparse tensor with 3 nonzeros.

X
```

```
sparse tensor of shape (3, 4, 2) with 3 nonzeros and order F
[0, 0, 0] = 1
[0, 1, 0] = 2
[2, 3, 1] = 3
```

CP-ALS

```
# Choose the rank and shape
R = 2
tensor_shape = (3, 4, 5)
```

```
# Set the random seed for reproducibility
np.random.seed(0)

# Create a low-rank dense tensor from a Kruskal tensor (i.e., ktensor)
factor_matrices = [np.random.randn(s, R) for s in tensor_shape]
M_true = ttb.ktensor(factor_matrices) # true solution
X = M_true.to_tensor()
```